
Semana 7 – Patrones de Comportamiento: Observer, Strategy, Command.

Objetivo

Implementar patrones de comportamiento que gestionan la comunicación entre objetos, optimizando interacción y desacoplamiento.

Marco Teórico

1. **Observer:** mantiene sincronización entre objetos (ej. sistemas de notificación).
 2. **Strategy:** selecciona algoritmos dinámicamente.
 3. **Command:** encapsula operaciones en objetos independientes, favoreciendo el undo/redo.
-

Observer

También llamado: Observador, Publicación-Suscripción, Modelo-patrón, Event-Subscriber, Listener

El patrón de diseño Observer (Observador) es un patrón de comportamiento que permite que un objeto notifique automáticamente a otros objetos cuando ocurre un cambio.

[Crear un sistema de notificaciones Observer para usuarios de un chat.](#)

Implementación en C++

```
#include <iostream>
#include <vector>
using namespace std;

// OBSERVER
class Usuario {
private:
    string nombre;
public:
    Usuario(string n) {
        nombre = n;
    }
    // Recibir notificación
    void actualizar(string mensaje) {
        cout << nombre << " recibio el mensaje: "
             << mensaje << endl;
    }
};

// SUBJECT
class Chat {
private:
    vector<Usuario*> usuarios;
```

```

public:

    // Agregar usuarios al chat
    void agregarUsuario(Usuario* usuario) {
        usuarios.push_back(usuario);
    }

    // Enviar mensaje a todos
    void enviarMensaje(string mensaje) {
        cout << "\nNuevo mensaje en el chat\n" << endl;
        for (int i = 0; i < usuarios.size(); i++) {
            usuarios[i]->actualizar(mensaje);
        }
    }
};

int main() {

    // Crear usuarios
    Usuario* u1 = new Usuario("Aldo");
    Usuario* u2 = new Usuario("Maria");
    Usuario* u3 = new Usuario("Carlos");

    // Crear chat
    Chat chat;

    // Agregar usuarios
    chat.agregarUsuario(u1);
    chat.agregarUsuario(u2);
    chat.agregarUsuario(u3);

    // Enviar mensajes
    chat.enviarMensaje("Hola a todos");
    chat.enviarMensaje("La reunion empieza a las 8 PM");

    return 0;
}

```

https://www.onlinegdb.com/edit/X_3QoGCb

SALIDA ESPERADA

Nuevo mensaje en el chat

Aldo recibio el mensaje: Hola a todos
 Maria recibio el mensaje: Hola a todos
 Carlos recibio el mensaje: Hola a todos

Nuevo mensaje en el chat

Aldo recibio el mensaje: La reunion empieza a las 8 PM
 Maria recibio el mensaje: La reunion empieza a las 8 PM
 Carlos recibio el mensaje: La reunion empieza a las 8 PM

Strategy

También llamado: Estrategia

Strategy: es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.

Usar Strategy para elegir diferentes algoritmos de ordenamiento (Burbuja, Quicksort).

IMPLEMENTACIÓN EN C++

```
#include <iostream>
#include <vector>
using namespace std;

// STRATEGY
class Ordenamiento {
public:
    virtual void ordenar(vector<int>& datos) = 0;
};

class Burbuja : public Ordenamiento {
public:

    void ordenar(vector<int>& datos) override {
        int n = datos.size();
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (datos[j] > datos[j + 1]) {
                    int temp = datos[j];
                    datos[j] = datos[j + 1];
                    datos[j + 1] = temp;
                }
            }
        }
        cout << "Ordenado con Burbuja" << endl;
    }
};

class QuickSort : public Ordenamiento {
private:

    void quicksort(vector<int>& datos, int inicio, int fin) {

        if (inicio >= fin) {
            return;
        }
        int pivote = datos[fin];
        int i = inicio - 1;

        for (int j = inicio; j < fin; j++) {
            if (datos[j] < pivote) {
                i++;
                int temp = datos[i];
                datos[i] = datos[j];
                datos[j] = temp;
            }
        }
    }
};
```

```

    }

    int temp = datos[i + 1];
    datos[i + 1] = datos[fin];
    datos[fin] = temp;

    int posicion = i + 1;

    quicksort(datos, inicio, posicion - 1);
    quicksort(datos, posicion + 1, fin);
}

public:

    void ordenar(vector<int>& datos) override {

        quicksort(datos, 0, datos.size() - 1);
        cout << "Ordenado con QuickSort" << endl;
    }
};

class SistemaOrdenamiento {
private:
    Ordenamiento* estrategia;
public:

    SistemaOrdenamiento(Ordenamiento* e) {
        estrategia = e;
    }
    void setEstrategia(Ordenamiento* e) {
        estrategia = e;
    }
    void ordenarDatos(vector<int>& datos) {
        estrategia->ordenar(datos);
    }
};

void mostrar(vector<int> datos) {
    for (int i = 0; i < datos.size(); i++) {
        cout << datos[i] << " ";
    }
    cout << endl;
}

int main() {

    vector<int> numeros = {5, 2, 9, 1, 7};

    // Estrategia Burbuja
    Burbuja b;

    SistemaOrdenamiento sistema(&b);
    sistema.ordenarDatos(numeros);
    mostrar(numeros);
    cout << endl;

    // Nuevos datos
    vector<int> numeros2 = {8, 4, 6, 3, 1};

    // Cambiar estrategia a QuickSort
    QuickSort q;

```

```
sistema.setEstrategia(&q);
sistema.ordenarDatos(numeros2);
mostrar(numeros2);

return 0;
}
```

<https://www.onlinegdb.com/edit/hDsLQiAdh>

SALIDA ESPERADA

```
Ordenado con Burbuja
1 2 5 7 9

Ordenado con QuickSort
1 3 4 6 8
```

Command

También llamado: Comando, Orden, Action, Transaction

Command: es un patrón de diseño de comportamiento que convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.

Implementar **Command** para un editor de texto con comandos “guardar” y “deshacer”.

IMPLEMENTACIÓN EN C++

```
#include <iostream>
#include <vector>
using namespace std;

class EditorTexto {
private:
    string texto;
public:
    void escribir(string nuevoTexto) {
        texto += nuevoTexto;
    }
    void guardar() {
        cout << "Texto guardado: " << texto << endl;
    }
    void deshacer() {
        if (texto.length() > 0) {
            texto.pop_back();
            cout << "Ultimo caracter eliminado" << endl;
        }
        else {
            cout << "No hay texto para deshacer" << endl;
        }
    }
}
```

```

        void mostrar() {
            cout << "Texto actual: " << texto << endl;
        }
};

class Comando {
public:
    virtual void ejecutar() = 0;
};

class ComandoGuardar : public Comando {
private:
    EditorTexto* editor;

public:
    ComandoGuardar(EditorTexto* e) {
        editor = e;
    }
    void ejecutar() override {
        editor->guardar();
    }
};

class ComandoDeshacer : public Comando {
private:
    EditorTexto* editor;

public:
    ComandoDeshacer(EditorTexto* e) {
        editor = e;
    }
    void ejecutar() override {
        editor->deshacer();
    }
};

class Boton {
private:
    Comando* comando;

public:
    Boton(Comando* c) {
        comando = c;
    }
    void presionar() {
        comando->ejecutar();
    }
};

int main() {

    // Receiver
    EditorTexto editor;
    editor.escribir("Hola Mundo");
    editor.mostrar();
    cout << endl;

    // Commands
    ComandoGuardar guardar(&editor);
    ComandoDeshacer deshacer(&editor);

    // Invokers
    Boton botonGuardar(&guardar);

```

```
    Boton botonDeshacer(&deshacer);

    // Ejecutar comandos
    botonGuardar.presionar();
    cout << endl;
    botonDeshacer.presionar();
    editor.mostrar();

    return 0;
}
```

<https://www.onlinegdb.com/edit/GbOLpD2ke4>

SALIDA ESPERADA

Texto actual: Hola Mundo

Texto guardado: Hola Mundo

Ultimo caracter eliminado

Texto actual: Hola Mund